INFORME TÉCNICO: MODELO LÓGICO, NORMALIZACIÓN Y DICCIONARIO DE DATOS GENÉRICO

Proyecto: Red Social Estudiantil Pascualina

Institución: Institución Universitaria Pascual Bravo

Módulo: Análisis y Diseño de Sistemas de Información / Bases de Datos I

Fecha: Septiembre de 2026

Estado: Entrega Final (Normalizado en 3FN)

1. Análisis de necesidades

El análisis de necesidades para la **Red Social Pascualina** responde a la urgencia de proporcionar a la comunidad estudiantil de la Institución Universitaria Pascual Bravo un entorno digital informal para la colaboración académica, la mentoría y la interacción social de pares . Las plataformas institucionales existentes resultan demasiado rígidas y formales, limitando la comunicación espontánea .

De los requerimientos del sistema se desprenden las siguientes acciones clave :

- **Creación y gestión de perfiles:** Registro de datos académicos (carrera, semestre, biografía) e intereses o habilidades técnicas y deportivas para impulsar la mentoría entre pares .
- **Red de conexiones horizontales:** Capacidad de seguir a otros compañeros en un esquema no jerárquico $M:N$ autorreferenciado .
- **Publicación en feed interactivo:** Compartir dudas, recursos académicos, noticias tecnológicas y memes .
- **Comunidades y grupos:** Creación de equipos para hackatones, grupos de estudio o clubes de pasatiempos .
- **Programación de eventos:** Coordinación de talleres y reuniones informales organizadas de forma independiente o patrocinadas por grupos .
- **Requerimientos de arquitectura y calidad de datos:**
    - **Diccionario de datos genérico:** Puente entre el modelo lógico y la implementación física (DDL) en PostgreSQL, MySQL o SQL Server.
    - **Tipos de datos y tamaño justo:** Asignación de tipos genéricos (*VARCHAR, INT, DATETIME, JSON*) con longitudes acotadas para evitar el desperdicio de almacenamiento en escenarios de Big Data e IoT.
    - **Restricciones de integridad:** Uso de no nulidad (NOT NULL), unicidad (UNIQUE) para correos y nombres de comunidades, y validaciones (CHECK) de rango.
    - **Datos semiestructurados:** Integración del tipo JSON para metadatos multimedia en publicaciones sin romper la Tercera Forma Normal (3FN).
    - **Versionado y trazabilidad:** Control de versiones por tabla y base de datos con capacidad de *Rollback* ante fallos.

2. Lluvia de ideas y mapa de conceptos

Se identificaron las entidades principales, sus atributos iniciales y sus interacciones de negocio antes de aplicar las reglas de normalización

Entidades preliminares

- **ESTUDIANTE:** Usuario central universitario .
- **HABILIDAD_INTERES:** Catálogo de disciplinas, tecnologías y deportes .
- **PUBLICACION:** Entradas de texto o recursos en el muro noticioso .
- **GRUPO:** Comunidades de estudio y clubes informales .
- **EVENTO:** Actividades, talleres y reuniones agendadas .

Matriz conceptual de relaciones e interacciones

Entidad Origen	Entidad Destino	Acción / Relación	Cardinalidad	Tipo de Relación
ESTUDIANTE	**ESTUDIANTE**	Conecta / Sigue	$M:N$	Autorreferenciada
ESTUDIANTE	**HABILIDAD_INTERES**	Posee / Le interesa	$M:N$	Requerirá tabla intermedia
ESTUDIANTE	**PUBLICACION**	Publica / Realiza	$1:N$	Directa
ESTUDIANTE	**GRUPO**	Crea / Administra	$1:N$	Creador responsable
ESTUDIANTE	**GRUPO**	Se une / Pertenece	$M:N$	Requerirá tabla intermedia
ESTUDIANTE	**EVENTO**	Organiza	$1:N$	Organizador principal

GRUPO	**EVENTO**	Patrocina / Aloja	$1:N$	Opcional
ESTUDIANTE	**EVENTO**	Confirma / Asiste	$M:N$	Requerirá tabla intermedia

3. Modelado Lógico y Diccionario de Datos

3.1 Referencia al modelo conceptual previamente aprobado

El modelo conceptual aprobado definió a **ESTUDIANTE** como el eje central de la red social . Para avanzar al modelo lógico, se refinaron las cardinalidades $M:N$ mediante tablas de intersección, alcanzando la Tercera Forma Normal (3FN) y garantizando la integridad de los datos

3.2 Correspondencias entre el modelo conceptual y el modelo lógico

Entidad Conceptual	Tabla(s) Lógica(s) Resultante(s)	Tipo de Transformación	Justificación Lógica
ESTUDIANTE	estudiante	Directa (1:1)	Entidad principal de usuarios
HABILIDAD_INTERES	habilidad_interes	Directa (1:1)	Catálogo unificado para evitar redundancias
(Relación Estudiante - Habilidad)	estudiante_habilidad	Nueva Tabla (Intersección)	Descompone la relación $M:N$ en dos $1:N$
(Relación Estudiante - Estudiante)	conexion_estudiante	Nueva Tabla (Intersección)	Descompone la autorreferencia $M:N$ de seguimiento .

PUBLICACION	publicacion	Directa (1:1)	Almacena entradas del feed noticioso
GRUPO	grupo	Directa (1:1)	Representa clubes o grupos de estudio
(Relación Estudiante - Grupo)	miembro_grupo	Nueva Tabla (Intersección)	Descompone la membresía $M:N$
EVENTO	evento	Directa (1:1)	Gestión de talleres y reuniones
(Relación Estudiante - Evento)	asistente_evento	Nueva Tabla (Intersección)	Descompone la confirmación de asistencia $M:N$

3.3 Nomenclatura, conversiones y normalización (1FN, 2FN, 3FN)

1. **Nomenclatura Standard:** Uso de minúsculas en convención snake_case (ej. id_estudiante, fecha_registro) y nombres de tabla en singular para asegurar compatibilidad entre motores SQL.
2. **Transformación $M:N$:** Toda relación Muchos a Muchos se convierte en una tabla de intersección cuya clave primaria es compuesta por las claves foráneas de las tablas participantes (PK, FK1, PK, FK2) .
3. **Paso por Formas Normales:**
    - **1FN:** Eliminación de campos multivaluados o listas separadas por comas
    - **2FN:** Todos los atributos no clave dependen de la totalidad de la clave primaria
    - **3FN:** Eliminación de dependencias transitivas (ej. se retira el nombre del autor de la tabla publicacion, dejando solo id_autor)

3.4 Control de versiones de la Base de Datos

Versión BD	Fecha	Responsable	Descripción del Cambio	Permite Rollback
1.0.0	2026-09-20	Equipo de Diseño	Creación inicial del modelo lógico y tablas base	Sí
1.1.0	2026-09-27	Equipo de Diseño	Ajuste de tablas descriptivas al referente formal, soporte JSON y reglas CHECK	Sí (Actual)

3.5 Descripción de atributos por tabla lógica (Diccionario de Datos)

Tabla Lógica: estudiante

- **Descripción:** Almacena los datos personales y académicos del perfil del estudiante
- **Versión de Tabla:** 1.1 | **Fecha:** 2026-09-27 | **Estimación:** ~376 Bytes / registro

Nombre del atributo	Tipo de dato y longitud	Descripción	Justificación de elección
id_estudiante	INT	Clave primaria (PK) autoincremental de la tabla	Se elige INT por ser un entero eficiente e idóneo para indexar búsquedas
nombre_completo	VARCHAR(100)	Nombres y apellidos completos del usuario	Se define VARCHAR(100) para acotar espacio sin cortar nombres largos

correo_institucional	VARCHAR(100)	Correo institucional único (UK, NOT NULL)	Se asigna VARCHAR(100) y unicidad para validar la identidad oficial
programa_academico	VARCHAR(80)	Carrera o programa de estudio actual	Se utiliza VARCHAR(80) para permitir filtrados por facultad
semestre	INT	Semestre académico activo (1 al 12)	Se elige INT con regla CHECK para garantizar datos dentro de rango
biografia	VARCHAR(255)	Breve resumen de presentación personal	Se define VARCHAR(255) opcional (NULL) para no bloquear el registro
fecha_registro	DATETIME	Estampa de fecha y hora de creación de la cuenta	Se elige DATETIME (NOT NULL) para trazabilidad e historial

Tabla Lógica: habilidad_interes

- **Descripción:** Catálogo normalizado de competencias técnicas, académicas y deportes
- **Versión de Tabla:** 1.0 | **Fecha:** 2026-09-27 | **Estimación:** ~90 Bytes / registro

Nombre del atributo	Tipo de dato y longitud	Descripción	Justificación de elección
id_habilidad	INT	Clave primaria (PK) del catálogo	Se elige INT autoincremental para búsquedas rápidas en el módulo de mentoría
nombre_habilidad	VARCHAR(50)	Nombre del tema o deporte (UK, NOT NULL)	Se asigna VARCHAR(50) con clave única para impedir duplicaciones de texto
tipo_categoria	VARCHAR(30)	Categoría general (Técnica, Deporte, Arte)	Se usa VARCHAR(30) para clasificar las búsquedas de los usuarios

Tabla Lógica: estudiante_habilidad

- **Descripción:** Tabla intermedia que conecta estudiantes con sus habilidades ($M:N$)
- **Versión de Tabla:** 1.0 | **Fecha:** 2026-09-27 | **Estimación:** ~40 Bytes / registro

Nombre del atributo	Tipo de dato y longitud	Descripción	Justificación de elección
id_estudiante	INT	Clave primaria y foránea (PK, FK1) a estudiante	Se elige INT para vincular al estudiante participante en la relación

id_habilidad	INT	Clave primaria y foránea (PK, FK2) a habilidad_interes	Se elige INT para asociar la competencia del catálogo
nivel_dominio	VARCHAR(20)	Grado de experiencia (Básico, Avanzado, Mentor)	Se asigna VARCHAR(20) para calificar el dominio del perfil

Tabla Lógica: conexion_estudiante

- **Descripción:** Tabla intermedia autorreferenciada para el seguimiento entre usuarios $(M:N)$.
- **Versión de Tabla:** 1.0 | **Fecha:** 2026-09-27 | **Estimación:** ~28 Bytes / registro

Nombre del atributo	Tipo de dato y longitud	Descripción	Justificación de elección
id_seguidor	INT	Clave primaria y foránea (PK, FK1) a estudiante .	Se elige INT para identificar al estudiante que realiza el seguimiento .
id_seguido	INT	Clave primaria y foránea (PK, FK2) a estudiante .	Se elige INT para identificar al estudiante receptor de la conexión .
fecha_conexion	DATETIME	Momento exacto en que inició la conexión	Se asigna DATETIME para ordenar cronológicamente las conexiones

Tabla Lógica: publicacion

- **Descripción:** Registro de mensajes, recursos y memes compartidos en el feed
- **Versión de Tabla:** 1.1 | **Fecha:** 2026-09-27 | **Estimación:** ~1058 Bytes / registro

Nombre del atributo	Tipo de dato y longitud	Descripción	Justificación de elección
id_publicacion	INT	Clave primaria (PK) de la publicación	Se elige INT autoincremental para identificación rápida de posts
id_autor	INT	Clave foránea (FK) vinculada a estudiante	Se asigna INT (NOT NULL) para garantizar la autoría del mensaje
contenido_texto	VARCHAR(1000)	Cuerpo o mensaje principal de la publicación	Se define VARCHAR(1000) para otorgar flexibilidad sin saturar memoria
tipo_publicacion	VARCHAR(30)	Categoría (Pregunta, Recurso, Noticia, Meme)	Se usa VARCHAR(30) con regla CHECK para permitir filtros en el feed
metadatos_adjuntos	JSON	Estructura flexible de archivos o enlaces	Se elige JSON para soportar adjuntos múltiples sin violar la 3FN
fecha_hora_publicacion	DATETIME	Fecha y hora de creación de la entrada	Se usa DATETIME para ordenar la

			secuencia temporal del feed

Tabla Lógica: grupo

- **Descripción:** Comunidades de estudio, equipos para hackatones y clubes
- **Versión de Tabla:** 1.0 | **Fecha:** 2026-09-27 | **Estimación:** ~373 Bytes / registro

Nombre del atributo	Tipo de dato y longitud	Descripción	Justificación de elección
id_grupo	INT	Clave primaria (PK) del grupo	Se elige INT autoincremental para identificación unívoca
nombre_grupo	VARCHAR(60)	Nombre oficial de la comunidad (UK, NOT NULL)	Se asigna VARCHAR(60) con clave única para evitar nombres duplicados
descripcion	VARCHAR(255)	Propósito y reglas generales del grupo	Se utiliza VARCHAR(255) para describir los objetivos a los miembros
tematica	VARCHAR(40)	Tema principal de interés (ej. Hackatones)	Se usa VARCHAR(40) para facilitar la búsqueda en el catálogo
id_creador	INT	Clave foránea (FK) vinculada a estudiante	Se elige INT (NOT NULL) para asignar la responsabilidad de administración

fecha_creacion	DATETIME	Estampa de tiempo de fundación del grupo	Se elige DATETIME para auditoría temporal

Tabla Lógica: miembro_grupo

- **Descripción:** Tabla intermedia de pertenencia de estudiantes a grupos (M: N)
- **Versión de Tabla:** 1.0 | **Fecha:** 2026-09-27 | **Estimación:** ~48 Bytes / registro

Nombre del atributo	Tipo de dato y longitud	Descripción	Justificación de elección
id_estudiante	INT	Clave primaria y foránea (PK, FK1) a estudiante	Se elige INT para identificar al estudiante integrante
id_grupo	INT	Clave primaria y foránea (PK, FK2) a grupo	Se elige INT para identificar al grupo correspondiente
rol_miembro	VARCHAR(20)	Rol asignado (Administrador, Miembro)	Se utiliza VARCHAR(20) para administrar permisos en la comunidad
fecha_union	DATETIME	Momento exacto de ingreso al grupo	Se asigna DATETIME para registro de antigüedad

Tabla Lógica: evento

- **Descripción:** Agenda de talleres, reuniones de estudio y actividades
- **Versión de Tabla:** 1.0 | **Fecha:** 2026-09-27 | **Estimación:** ~758 Bytes / registro

Nombre del atributo	Tipo de dato y longitud	Descripción	Justificación de elección

id_evento	INT	Clave primaria (PK) del evento	Se elige INT autoincremental como identificador unívoco
titulo_evento	VARCHAR(80)	Nombre corto de la actividad	Se define VARCHAR(80) para su despliegue claro en calendarios
descripcion	VARCHAR(500)	Agenda y detalle completo de la reunión	Se asigna VARCHAR(500) para proveer la información detallada
fecha_hora_inicio	DATETIME	Programación temporal de inicio del evento	Se elige DATETIME para coordinar alertas y notificaciones
lugar_o_enlace	VARCHAR(150)	Ubicación física o enlace de videoconferencia	Se utiliza VARCHAR(150) para soportar actividades presenciales o virtuales
id_organizador	INT	Clave foránea (FK) vinculada a estudiante	Se elige INT (NOT NULL) para fijar al estudiante responsable

id_grupo_asociado	INT	Clave foránea (FK) opcional hacia grupo	Se elige INT (NULL) para permitir eventos individuales o de grupo

Tabla Lógica: asistente_evento

- **Descripción:** Tabla intermedia de confirmación de asistencia a eventos ($M:N$)
- **Versión de Tabla:** 1.0 | **Fecha:** 2026-09-27 | **Estimación:** ~48 Bytes / registro

Nombre del atributo	Tipo de dato y longitud	Descripción	Justificación de elección
id_estudiante	INT	Clave primaria y foránea (PK, FK1) a estudiante	Se elige INT para enlazar al estudiante asistente
id_evento	INT	Clave primaria y foránea (PK, FK2) a evento	Se elige INT para enlazar al evento programado
estado_asistencia	VARCHAR(20)	Estado (Confirmado, Pendiente, Cancelado)	Se asigna VARCHAR(20) con CHECK para control de aforo
fecha_confirmacion	DATETIME	Marca de tiempo de la respuesta del usuario	Se elige DATETIME para auditoría logística

3.6 Descripción de las relaciones del modelo lógico

Relación: Estudiante Posee Habilidad

Nombre de la Relación	Tablas involucradas	Cardinalidad	Implementación lógica propuesta	Justificación de la implementación
Estudiante Posee Habilidad	estudiante / habilidad_interes	$M:N$ (Un Estudiante posee N Habilidades y una Habilidad pertenece a N Estudiantes)	Se crea la tabla intermedia estudiante_habilidad con clave compuesta (id_estudiante, id_habilidad).	Descompone la relación $M:N$ en dos $1:N$, permitiendo asociar múltiples competencias por perfil y agregar el calificador nivel_dominio

Relación: Estudiante Sigue Estudiante

Nombre de la Relación	Tablas involucradas	Cardinalidad	Implementación lógica propuesta	Justificación de la implementación
Estudiante Sigue Estudiante	estudiante (Seguidor) / estudiante (Seguido)	$M:N$ (Un Estudiante sigue a N Estudiantes y es seguido por N Estudiantes)	Se crea la tabla de intersección conexion_estudiante con clave compuesta (id_seguidor, id_seguido).	Resuelve la autorreferencia $M:N$ sin duplicar tablas de usuarios, evitando registros repetidos de seguimiento mediante la clave primaria compuesta .

Relación: Estudiante Publica Post

Nombre de la Relación	Tablas involucradas	Cardinalidad	Implementación lógica propuesta	Justificación de la implementación
Estudiante Publica Post	estudiante / publicacion	$1:N$ (Un Estudiante tiene N Publicaciones)	Se añade el atributo id_autor de tipo INT (FK) a la tabla publicacion.	Para que cada publicación esté inequívocamente vinculada a su autor, la tabla publicacion debe referenciar al estudiante, permitiendo consultar su historial .

Relación: Estudiante Administra Grupo

Tablas Involucradas	Cardinalidad	Implementación lógica propuesta	Justificación de la implementación
Estudiante Administra Grupo	estudiante / grupo	$1:N$ (Un Estudiante administra N Grupos)	Se añade el atributo id_creador de tipo INT (FK) a la tabla grupo.

Relación: Estudiante Pertenece A Grupo

Nombre de la Relación	Tablas involucradas	Cardinalidad	Implementación lógica propuesta	Justificación de la implementación

Estudiante Pertenece A Grupo	estudiante / grupo	$M:N$ (Un Estudiante pertenece a N Grupos y un Grupo tiene N Miembros)	Se crea la tabla intermedia miembro_grupo con clave compuesta (id_estudiante, id_grupo).	Descompone la relación $M:N$, permitiendo gestionar el listado de miembros y asignar el atributo rol_miembro dentro del grupo

Relación: Estudiante Organiza Evento

Nombre de la Relación	**Tablas involucradas**	**Cardinalidad**	**Implementación lógica propuesta**	**Justificación de la implementación**
Estudiante Organiza Evento	estudiante / evento	$1:N$ (Un Estudiante organiza N Eventos)	Se añade el atributo id_organizador de tipo INT (FK) a la tabla evento.	Establece de forma directa al estudiante líder de la reunión o taller, siendo responsable de la logística

Relación: Grupo Patrocina Evento

Nombre de la Relación	**Tablas involucradas**	**Cardinalidad**	**Implementación lógica propuesta**	**Justificación de la implementación**
Grupo Patrocina Evento	grupo / evento	$0..1:N$ (Un Grupo puede patrocinar N Eventos)	Se añade el atributo id_grupo_asociado de tipo INT (FK, NULL) a la tabla evento.	Permite que un evento pertenezca opcionalmente a un grupo. Al admitir nulos (NULL), soporta también eventos independientes

Relación: Estudiante Confirma Evento

Nombre de la Relación	Tablas involucradas	Cardinalidad	Implementación lógica propuesta	Justificación de la implementación
Estudiante Confirma Evento	estudiante / evento	$M:N$ (Un Estudiante asiste a N Eventos y un Evento registra N Asistentes)	Se crea la tabla intermedia asistente_evento con clave compuesta (id_estudiante, id_evento).	Permite controlar el aforo de las actividades y llevar el registro individual de respuestas mediante estado_asistencia

4. Presentación y retroalimentación

mi Retroalimentación personal

- **Adopción de JSON:** Se aprobó el uso del tipo genérico JSON en publicacion.metadatos_adjuntos, permitiendo guardar múltiples enlaces e imágenes sin romper la 3FN ni cargar el sistema con tablas extra
- **Dimensionamiento contra Big Data:** Se sustentó la reducción de campos a VARCHAR(100) y VARCHAR(60) en lugar del valor genérico 255. Esto optimiza el consumo de RAM e I/O en discos cuando la plataforma alcance alto volumen de usuarios
- **Estructura de tablas descriptivas:** Se ajustaron las tablas al referente institucional de 4 columnas para atributos y 5 columnas para relaciones, facilitando la lectura ejecutiva y la traducción física a SQL
- **Control de versiones y validadores:** Se validó la bitácora de versiones con capacidad de *Rollback* y la inclusión de reglas CHECK

5. Versión final y entrega del producto

Memoria justificativa

El modelo lógico y diccionario de datos genérico de la **Red Social Pascualina** ofrece una estructura relacional limpia en Tercera Forma Normal (3FN), optimizada para su implementación física inmediata mediante DDL

1. **Eficiencia y 3FN:** La eliminación de atributos multivaluados mediante las tablas asociativas estudiante_habilidad, conexion_estudiante, miembro_grupo y asistente_evento garantiza cero redundancia
2. **Standard de diseño:** Convención snake_case homogénea, claves primarias (PK), foráneas (FK), unívocas (UK) y validaciones de rango CHECK

3. **Escalabilidad:** El acotamiento de tipos de datos y la capacidad de auditoría con versionado aseguran una solución robusta y preparada para producción

Conclusiones individuales sobre el proceso de solución

- **Aspectos relevantes:** Comprender el valor metodológico de construir un Diccionario de Datos Genérico antes de escribir código en el motor SGBD fue el aprendizaje más relevante Ajustar las tablas al referente formal permitió estructurar la entrega con rigor técnico
- **Dificultades encontradas:** Modelar el seguimiento entre estudiantes representó el mayor reto . Se solucionó mediante la relación autorreferenciada $M:N$ implementada con la tabla de intersección conexion_estudiante .
- **Dudas resueltas:** Se comprendió la diferencia práctica entre claves primarias (PK) y claves únicas de negocio (UK), así como el uso estratégico de datos semiestructurados JSON para integrar la flexibilidad no relacional dentro del modelo relacional normalizado